

PKCERT AI & Software Development Internship

Task 20 – Build and Train a Simple Feedforward Neural Network on MNIST (or a Similar Dataset)

Objective: Having implemented a feed-forward network with `nn.Module` and `autograd` in Task 18, and having built loss functions, optimizers, and the training-loop mechanics from scratch in Task 19, interns will now assemble these components into a single, complete, end-to-end training pipeline on MNIST (or a stated similar dataset, such as Fashion-MNIST or KMNIST). The network architecture itself is intentionally simple — fully connected layers only, no convolutions — so the difficulty of this task lies in engineering and evaluation rigor: a correct train/validation/test split, leakage-free preprocessing, an honest evaluation methodology, a controlled ablation, and full reproducibility documentation. Every reported number must come from your own run, not from memory or from a tutorial's stated results.

Total Marks: 100

Difficulty Level: Hard

Part A – Data Pipeline & Preprocessing (20 Marks)

- Load MNIST (or a similar dataset — Fashion-MNIST or KMNIST) using `torchvision.datasets`, and implement a proper three-way train/validation/test split (not just train/test), stating the split ratios and a fixed random seed for reproducibility.
- Compute normalization statistics (mean and standard deviation) from the training split only, and explain why computing them from the full dataset, including validation or test data, would constitute data leakage.
- Flatten each 28×28 image into a 784-dimensional vector for the fully-connected architecture, and print tensor shapes at each pipeline stage (raw image, flattened vector, batched tensor) as a sanity check.
- Wrap the data in `torch.utils.data.Dataset` and `DataLoader` objects, visualize a sample batch with its labels, and report the class distribution across the train, validation, and test splits to confirm the classes are reasonably balanced.

Part B – Model Architecture & Design Justification (20 Marks)

- Design a simple feed-forward network (2–4 fully connected layers) using `nn.Module`; state the chosen hidden-layer widths and justify them relative to the 784-dimensional input and 10-class output.
- Justify the choice of activation function (e.g., ReLU vs. sigmoid vs. tanh) with a short mathematical explanation of how ReLU's gradient behaves differently from sigmoid/tanh once those functions saturate, and why this matters as networks get deeper.
- Justify the output-layer and loss-function pairing (raw logits with `CrossEntropyLoss` vs. softmax with `NLLLoss`), confirm numerically that both formulations produce the same loss value on a sample batch, and explain why the logits + `CrossEntropyLoss` form is numerically preferred (log-sum-exp stability).
- Calculate the model's total trainable parameter count by hand from the layer dimensions, and verify the hand calculation against `sum(p.numel() for p in model.parameters())`.

Part C – Training, Evaluation & Experiments (40 Marks)

- Train the network end-to-end using an optimizer, loss function, and training-loop structure consistent with Task 19; state and justify the chosen optimizer, learning rate, batch size, and number of epochs.
- Track training loss, validation loss, and validation accuracy at every epoch, plot all three on a single figure, and identify the epoch at which validation loss stops improving (i.e., where overfitting begins, if it does).
- Report final test-set accuracy, precision, recall, and macro-averaged F1 score, and generate a confusion matrix; identify the class pair(s) most frequently confused and give a plausible explanation grounded in the confusion matrix itself.
- Run a controlled ablation comparing at least two configurations (e.g., with vs. without a hidden layer, two different hidden-layer widths, or two learning rates), holding all other hyperparameters and the random seed fixed, and report the resulting difference in test accuracy.
- Save the trained model's parameters with `torch.save(model.state_dict(), ...)`, reload them into a freshly-initialized model instance, and confirm the reloaded model's predictions exactly match the original model's predictions on a held-out batch.

Part D – Analysis & Documentation (20 Marks)

- Summarize the final architecture, hyperparameters, and evaluation metrics (accuracy, precision, recall, F1) in a single results table.
- Include the confusion matrix and the loss/accuracy curves as figures, each with a short written interpretation.
- Reflect on one counter-intuitive or unexpected finding from the Part C ablation (e.g., a wider hidden layer not improving accuracy, or an added layer overfitting faster) and explain the likely mechanism behind it.
- Describe one concrete implementation issue encountered while building the pipeline (e.g., a data-leakage bug in normalization, a shape mismatch during flattening, or a label/prediction misalignment in the confusion matrix), how it was diagnosed, and how it was fixed.
- Document the PyTorch and torchvision versions used, all hyperparameters, and the random seed(s) fixed for each run, so that the results are reproducible.

Assessment Breakdown

Assessment Area	Marks
Data Pipeline & Preprocessing	20
Model Architecture & Design Justification	20
Training, Evaluation & Experiments	40
Analysis & Documentation	20
Total	100

Note: This task assembles the components built in Task 18 (network architecture) and Task 19 (training-loop mechanics) into a single, complete, and properly evaluated training pipeline. Although the network itself is intentionally simple — fully connected layers only — the emphasis is on rigor: a genuine train/validation/test split (not just train/test), leakage-free preprocessing, an honest evaluation methodology, and full reproducibility documentation. Report the dataset used (MNIST or the stated alternative), the

PyTorch/torchvision version, all hyperparameters, and the random seed(s) fixed for each experiment, so that results are reproducible.